

TechnoSky Workflow Guide

23rd February, 2026

PANAROMICA

Questa guida spiega come creare e strutturare correttamente i flussi di lavoro di base in Creator in modo che i tecnici di Technosky possano configurarli e riutilizzarli in modo indipendente.

SCOPRO

Lo scopo di questa guida è definire la struttura e le regole necessarie per creare modelli di workflow riutilizzabili in Creator, garantendo una configurazione coerente e autonoma da parte dei tecnici Technosky.

PREREQUISTI

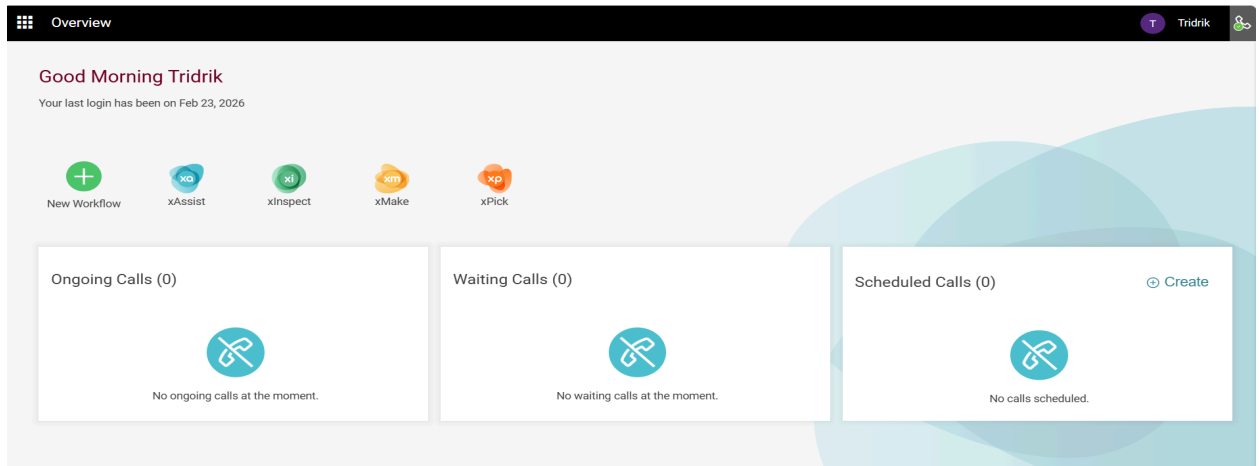
Accesso a Frontline Creator: È necessario avere accesso a un'istanza del server con un utente che abbia i ruoli di Creator e Developer.

PASSI PER IL FLUSSO DI LAVORO DI BASE

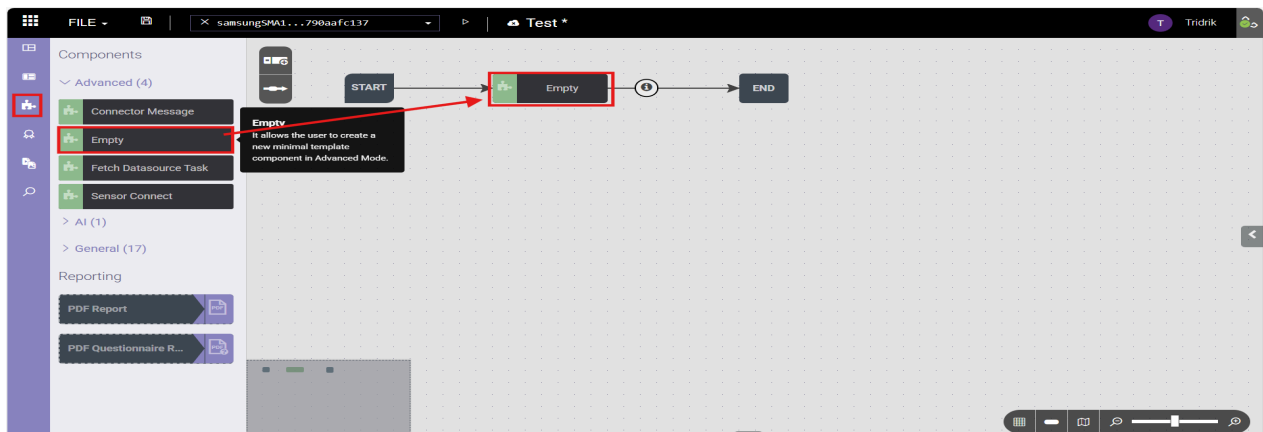
1. Accedi con il tuo nome utente e la tua password



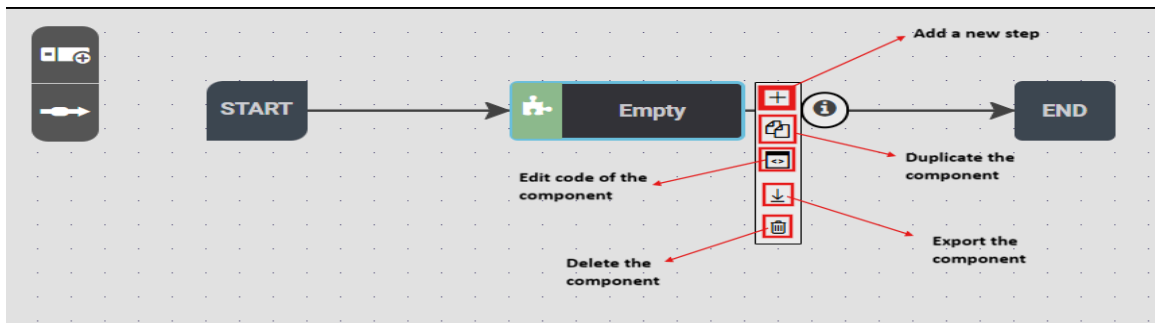
2. Si apre la Dashboard. Clicca su Nuovo Flusso di Lavoro.



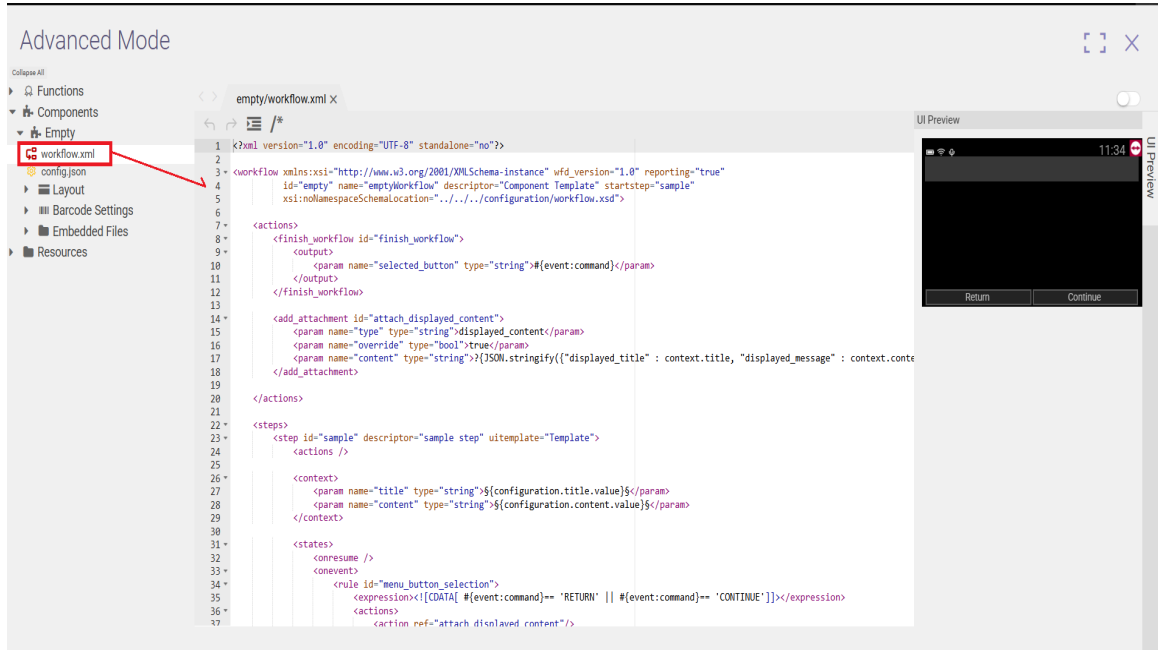
3. Inserisci il nome del tuo flusso(NEW WORKFLOW) di lavoro e invialo.
4. Trascina il componente EMPTY e posizionalo sul pannello vuoto.



5. Quindi collega le frecce da START al componente EMPTY, quindi dal componente EMPTY a END trascinando il cursore.
6. Fai clic sul EMPTY COMPONENT, appare un pannello di scorciatoie.



7. Clicca su Modifica Codice per aprire la Modalità Avanzata, dove è possibile modificare il flusso di lavoro.



8. Un tipico telaio strutturale di un componente è il seguente:

```
<workflow [ATTRIBUTES]>
  <context> [...] </context>
  <actions> [...] </actions>
  <rules>
    <rule id="menu\_button\_selection">
      <expression>[...]</expression>
      <actions>
        [...]
      </actions>
    </rule>
  </rules>

  <steps>
    <step id="stepA" [ATTRIBUTES]>
      <context> [...] </context>
      <rules> [...] </rules>
      <actions> [...] </actions>
      <states>
        <onevent>
          <rule ref="menu\_button\_selection"/>
          <rule id="show\_notification">
            <expression>[...]</expression>
            <actions>
              [...]
            </actions>
          </rule>
        </onevent>
      </states>
    </step>

    <step id="stepB" [ATTRIBUTES]>
      <states>
        <onevent>
          <rule ref="menu\_button\_selection"/>
        </onevent>
      </states>
    </step>
  </steps>
</workflow>
```

Workflow scope

Step scope

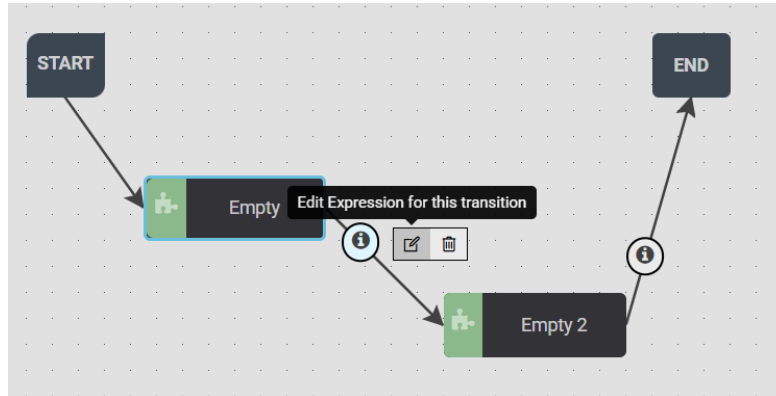
Referencing a rule from the workflow scope

Direct definition (could still reference a pre-defined action)

Referencing a rule from the workflow scope

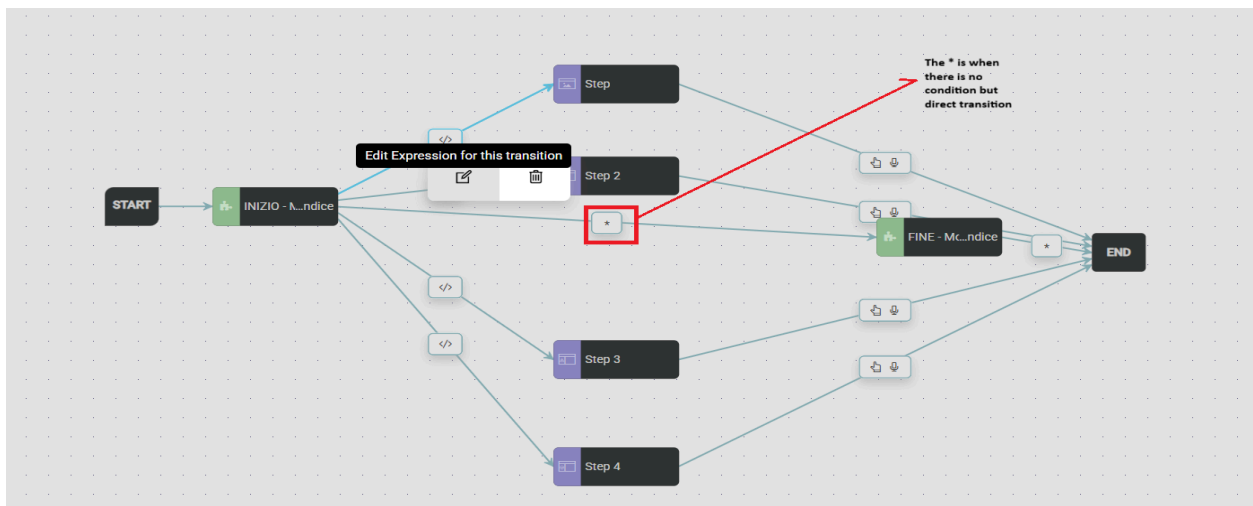
Per ottenere l'intera banca dati delle conoscenze, click: [Creator Developer Guide](#) and [FrontLine Command Center](#).

9. Ogni blocco componente sarà collegato da frecce in sequenza. I loro editor di espressione dovrebbero contenere la logica per la transizione da un componente all'altro.



all'altro.

OPPURE



10. Clicca sull'espressione da modificare, si apre l'editor. Clicca su ADD RULE per la Transizione.

Expression Editor

Component Transition

NOT AND OR

Select context param

Remote Command Remote Command

extbcreader.NUM_CONNECTED_DEVICES1

extdevices.connection_statusCONNECTED

reportingEnabledtrue

Scanned BarcodeScanned Barcode

selected_button#(event:clicked)

+ ADD RULE

+ ADD GROUP

11. Seleziona il context param → equals/exists/not exists → true/false→ Save

Expression Editor ✕

Component Transition

▼ ☐ NOT AND OR

+ ADD RULE ☐ ADD GROUP

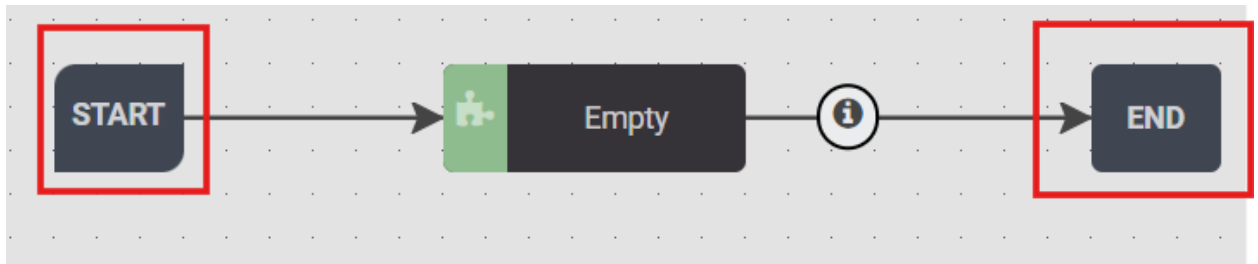
reportingEnabled ▼

equal ▼

▼

true
false

12. Da notare: La catena di componenti collegati dalle frecce con le loro espressioni di transizione deve iniziare da START e terminare a END, per completare il flusso di lavoro.



“START WORKFLOW” LOGIC OF TECHNOSKY

PANORAMICA DEL FLUSSO DI LAVORO

Workflow Identifier

id="selection_site" startstep="home"

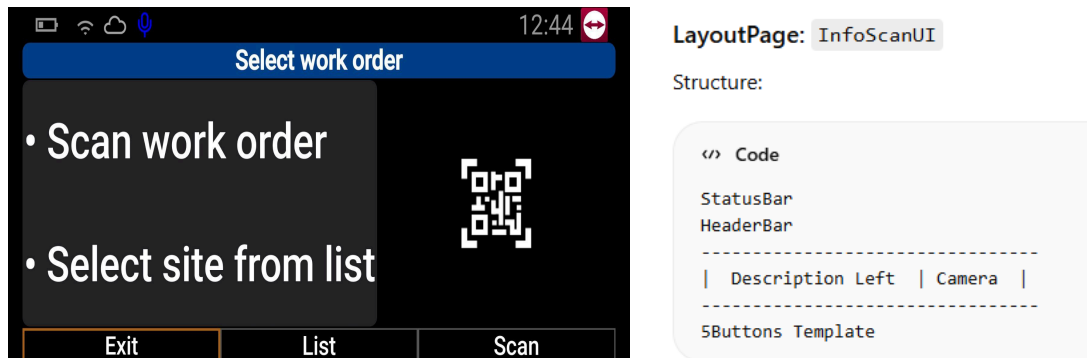
```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<workflow
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" wfd_version="1.0"
  id="selection_site" name="selection_site" descriptor="Selection site" startstep="home"
  xsi:noNamespaceSchemaLocation="../../../../configuration/workflow.xsd">
```

Questo flusso di lavoro gestisce: Work order selection/Scan or list selection/Backend order request/ Dynamic template loading via Job Plan ID

2. FLUSSO TECNICO PASSO DOPO PASSO

STEP 1 — **home** UI Template Used: uitemplate="InfoScanUI"

Screenshot collegato:



Schermata che mostra:

- Header: **Select work order**
- Opzioni:
 - Scan work order
 - Select site from list
- Buttons:
 - Exit
 - List
 - Scan

Dynamic Camera Overlay:

```
<StackItem Name="CameraLayer" Visibility="Invisible" Orientation="Horizontal">
  <Camera Weight="1">
    <MaskOverlay/>
    <RectangleOverlay/>
    <AROverlay/>
  </Camera>
</StackItem>
```

La fotocamera è nascosta di default. Quando Scan è premuto, viene attivata utilizzando il comando vocale:

- CameraLayer → Visible
- Barcode scanner starts

Configurazione dei pulsanti

I pulsanti sono configurati tramite:

```
<ui_update id="show_buttons">
  <widget_params>
    <ui_element name="1BTN">
      <param name="visibility">visible</param>
    </ui_element>
    <ui_element name="1BTN_TEXT">
      <param name="content">${button.exit}</param>
    </ui_element>
    <ui_element name="2BTN">
      <param name="visibility">visible</param>
    </ui_element>
    <ui_element name="2BTN_TEXT">
      <param name="content">${button.list}</param>
    </ui_element>
    <ui_element name="3BTN">
      <param name="visibility">visible</param>
    </ui_element>
    <ui_element name="3BTN_TEXT">
      <param name="content">${button.scan}</param>
    </ui_element>
  </widget_params>
</ui_update>
```

Mapping:

Pulsanti	Nome Interno	Funzioni
Exit	1BTN	Exit workflow
List	2BTN	Open order list
Scan	3BTN	Activate camera

2.3 Event Rules — home

Uscita <code>#{event:command} == '1BTN'</code> Inneschi: <code><finish_workflow id="wf_exit"/></code>	List <code>#{event:command} == '2BTN'</code> Sets: <code>start_from_list = true</code> Inneschi: <code>wf_list</code>
<pre><rule id="exit"> <expression> <![CDATA[#{event:command} == '1BTN']]> </expression> <actions> <action ref="wf_exit"/> </actions> </rule></pre>	<pre><rule id="list"> <expression> <![CDATA[#{event:command} == '2BTN']]> </expression> <actions> <setvar id="set_start_from"> <context_of>root</context_of> <context_update> <param name="start_from_list" type="bool">true</param> <param name="start_from_scan" type="bool">false</param> </context_update> </setvar> <action ref="wf_list"/> </actions> </rule></pre>

Attivazione della scansione <code>{event:command} == '3BTN' && !({camera_active})</code>	Chiudi scansione <code>{event:command} == '3BTN' && {camera_active}</code>
Azioni: • show_camera • start_camera • set_camera_active = true	Azioni: • hide_camera • stop_camera • set_camera_inactive
<pre> <rule id="scan"> <expression> <![CDATA[{event:command} == '3BTN' && !({camera_active})]]> </expression> <actions> <action ref="show_camera"/> <action ref="start_camera"/> <action ref="set_camera_active"/> </actions> </rule> </pre>	<pre> <rule id="close_scan"> <expression> <![CDATA[{event:command} == '3BTN' && {camera_active}]]> </expression> <actions> <action ref="hide_camera"/> <action ref="stop_camera"/> <action ref="set_camera_inactive"/> </actions> </rule> </pre>

Estrazione del codice a barre Quando viene estratto un codice a barre valido: <code>{event(value_extractor):command} == 'VALID_EXTRACTION'</code>	<pre> <rule id="validation_correct"> <expression><![CDATA[{event(value_extractor):command} == 'VALID_EXTRACTION'}]]></expression> <actions> <action ref="hide_camera"/> <action ref="stop_camera"/> <action ref="set_camera_inactive"/> <setvar id="set_scanned_workzone_or_zz"> <context_of>root</context_of> <context_update> <param name="scanned_workzone_or_zz" type="string">#{scanned}</param> </context_update> </setvar> <action ref="go_to_request"/> </actions> </rule> </pre>
Sistema: 1. Ferma la fotocamera 2. Memorizza il valore scansionato: scanned_workzone_or_zz = <code>#{scanned}</code> 3. Passa a: go_to_request	<pre> <context_of>root</context_of> <context_update> <param name="scanned_workzone_or_zz" type="string">#{scanned}</param> </context_update> </setvar> <action ref="go_to_request"/> </pre>

STEP 2 — request UI Template:WaitingUI

```
<step id="request" uitemplate="WaitingUI" persist="false" screen_lock_enabled="true" debug_mode="false">
```

Screenshot connesso:

- Animazione del caricatore
- “Attesa del server”
- Finestra di dialogo per ritentare in caso di fallimento

Backend Call

On enter:

```
<send_commit_message id="request_inspection_order">
```

```
<actions>
  <send_commit_message id="request_inspection_order" type="{Connector.type1.value}" connection_timeout="{Connector.connectionTimeout.value}">
    <properties>
      <param name="order_code" type="string">#{scanned_workzone_or_zz}</param>
      <param name="is_scanned" type="string">true</param>
      <param name="user_id" type="string">#{user_id}</param>
      <param name="event_handler" type="string">#{event_handler_type}</param>
    </properties>
  </send_commit_message>
</actions>
```

Parametri Inviati:

Parametri	Valore
order_code	scanned_workzone_or_zz
is_scanned	true
user_id	current user
event_handler	handler type

Gestione delle risposte:

Condizione di successo RESPONSE_OK && payload.response == 'OK' AND order_details not empty. Azioni: 1. Store full order_details object 2. Set is_order_header = true 3. Set start_from_scan = true 4. Transition to: go_to_set_parameter	<pre><rule id="response_ok"> <expression> <![CDATA[#{event(send_commit_message):command} == 'RESPONSE_OK' && #{event(send_commit_message):payload.response} == 'OK' && ?{Object.keys(context.order_details).length > 0 }?]]> </expression> </actions> </rule> <rule id="response_ok_0"> <expression> <![CDATA[#{event(send_commit_message):command} == 'RESPONSE_OK' && #{event(send_commit_message):payload.response} == 'OK' && ?{!(Object.keys(context.order_details).length > 0)}?]]> </expression> </actions> </rule></pre>
Condizioni di errore Gestito: ● RESPONSE_TIMEOUT ● RESPONSE_ERROR ● RESPONSE_NOT_OK ● Empty order_details Tutti i trigger: ● negative sound ● retry dialog	<pre><rule id="response_not_ok"> <expression>... </expression> </actions> <action ref="negative_feedback_sound" /> <action ref="show_retry" /> </actions> </rule> <rule id="response_timeout"> <expression> <![CDATA[#{event(send_commit_message):command} == 'RESPONSE_TIMEOUT']]> </expression> </actions> </rule> <rule id="response_error"> <expression> <![CDATA[#{event(send_commit_message):command} == 'RESPONSE_ERROR']]> </expression> </actions> </rule> <rule id="dialog_retry"> <expression>#{event:command} == 'RETRY'</expression> </actions> </rule></pre>

Questo è un passaggio di elaborazione.

Inizializzazione del parametro d'ordine

Attivato all'inserimento: init_order_params init_order_notes	<pre><onenter> <rule id="init"> <expression>1</expression> <actions> <action ref="init_order_params"/> <action ref="init_order_notes"/> </actions> </rule> </onenter></pre>
init_order_params	Popola il contesto principale con: • Informazioni sull'ordine di lavoro • Informazioni sul piano di lavoro • Informazioni sull'asset • Informazioni sulla manutenzione preventiva (PM) • Informazioni sul WOC • Timestamp di esecuzione • Date formattate

Importante: La formattazione delle date viene gestita all'interno dei blocchi JavaScript CDATA.
Esempio: `var raw = context.order_details.order.schedstart;`
Convertito in: `gg/mm/aa hh:mm:ss`

Dynamic Workflow Extraction

On resume: get_workflow_from_template	<pre><onresume> <rule id="auto"> <expression>1</expression> <actions> <action ref="set_order_tutorial_params_true"/> <action ref="get_workflow_from_template"/> </actions> </rule> </onresume></pre>
Nome del modello: <code>#{order_jpnum}</code>	<pre><get_workflow_from_template id="get_workflow_from_template"> <param name="template">#{order_jpnum}</param> </get_workflow_from_template></pre> Questo è il numero del Piano di Lavoro.

Gestione dei Risultati dell'Estrazione

Se il modello esiste	<pre><rule id="extracted"> <expression><![CDATA[#{event:command} == 'WORKFLOW_EXTRACTED']]></expression> <actions> <action ref="set_order_tutorial_params_true"/> <action ref="wf_scan_order"/> </actions> </rule></pre>
Evento: WORKFLOW_EXTRACTED	Azioni: - tutorial = true - wf_scan_order triggered

Se il modello non esiste	<pre><rule id="extraction_error"> <expression><![CDATA[#{event:command} == 'WORKFLOW_EXTRACTION_ERROR']]></expression> <actions> <action ref="set_order_tutorial_params_false"/> <action ref="wf_scan_order"/> </actions> </rule></pre>
Evento: WORKFLOW_EXTRACTION_ERROR	Azioni: - tutorial = false - wf_scan_order triggered

SOMMARIO DELLA LOGICA DEL TEMPLATE DINAMICO

Il flusso di lavoro carica dinamicamente un template se:

Template name == order_jpnum

Se corrisponde:	- Il pulsante della guida appare - Il flusso di lavoro personalizzato viene inserito
Se nessuna corrispondenza:	Il flusso normale continua

RIEPILOGO DELLA CONNETTIVITÀ DELL'INTERFACCIA UTENTE

Step	UI Template	Scopo
home	InfoScanUI	Selezione dell'ordine di lavoro
request	WaitingUI	Richiesta backend
set_parameter	WaitingUI	Richiesta backend Preparazione dei dati + estrazione dinamica dei modelli

7. COMPORTAMENTO DELLA TELECAMERA

```
<camera_scan_barcode id="start_camera">
  <settings>${generic_scanner.xml}</settings>
</camera_scan_barcode>
<stop_camera id="stop_camera"/>
<ui_update id="show_camera">
  <widget_params>...
</widget_params>
</ui_update>
<ui_update id="hide_camera">
  <widget_params>...
</widget_params>
</ui_update>
<get_workflow_from_template id="get_workflow_from_template">
  <param name="template">#{order_jpnum}</param>
</get_workflow_from_template>
<setvar id="set_camera_active">
  <context_of>step</context_of>
  <context_update>
    <param name="camera_active" type="bool">true</param>
  </context_update>
</setvar>
<setvar id="set_camera_inactive">
  <context_of>step</context_of>
  <context_update>
    <param name="camera_active" type="bool">false</param>
  </context_update>
</setvar>
```

Stato della fotocamera controllato da:	camera_active (boolean)
Overlay controllato da:	show_camera / hide_camera
Timeout gestito tramite:	TIMEOUT_REACHED

8. VARIABILI DI CONTESTO CREATE

Variabili create: • order_details (object) • order_jpnum • order_wonum • order_assetnum • order_notes (array) • tutorial (boolean) • tutorial_uri • start_from_scan • start_from_list Questi sono accessibili a livello globale nei flussi di lavoro successivi.	<pre><context_of>root</context_of> <context_update> <!-- General --> <param name="order_previous_user_id" type="string">?{context <param name="order_haltocreview" type="string">?{context <param name="order_href_path" type="string">?{context <!-- Workorder --> <param name="order_wonum" type="string">?{context.orde <param name="order_wodesc" type="string">?{context.orde <param name="order_wostatus" type="string">?{context.orde <param name="order_woschedstart" type="string"><![CDATA[?{ <param name="order_halistoc" type="string">?{context.order_detail <param name="order_halistoc_to_show" type="string">?{context.orde <!-- Job Plan --> <param name="order_jpnum" type="string">?{context.order_detail <param name="order_jpdesc" type="string">?{context.order_detail <!-- Asset --> <param name="order_assetnum" type="string">?{context.order_detail <param name="order_assetdesc" type="string">?{context.order_detail <param name="order_haltechid" type="string">?{context.order_detail <param name="order_parentasset" type="string">?{context.order_detail <!-- Execution --> <param name="order_actstart" type="string"><![CDATA[?{ <param name="order_actfinish" type="string">?{context.order_details.exe <!-- PM / Frequency --> <param name="order_pmnum" type="string">?{context.order_details.pm. <param name="order_pmdesc" type="string">?{context.order_details.pm. <param name="order_freqpolicynum" type="string">?{context.order_details.pm. <param name="order_freqpolicydesc" type="string">?{context.order_details.pm. <!-- WOC --> <param name="order_halwoc" type="string">?{context.order_details.woc <param name="order_halwocyear" type="string">?{context.order_details.woc <param name="order_halwocauth" type="string">?{context.order_details.woc</pre>
--	--

TRANSIZIONI CONTROLLATE

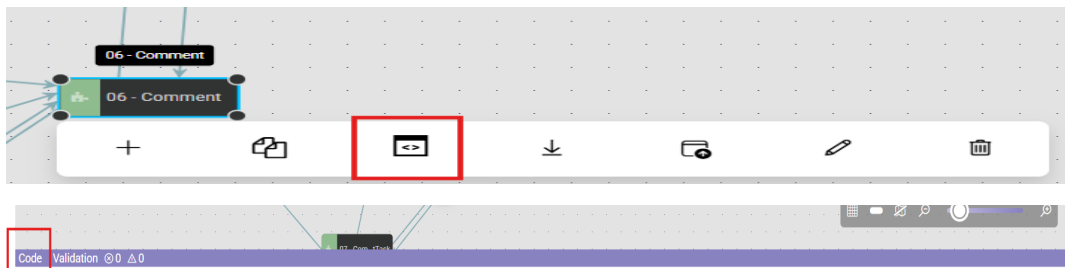
Transizioni definite:	go_to_home go_to_request go_to_set_parameter Nessuna navigazione implicita. Tutte le transizioni devono essere dichiarate esplicitamente.
------------------------------	---

SOMMARIO DEL COMPORTAMENTO DEL FLUSSO DI LAVORO

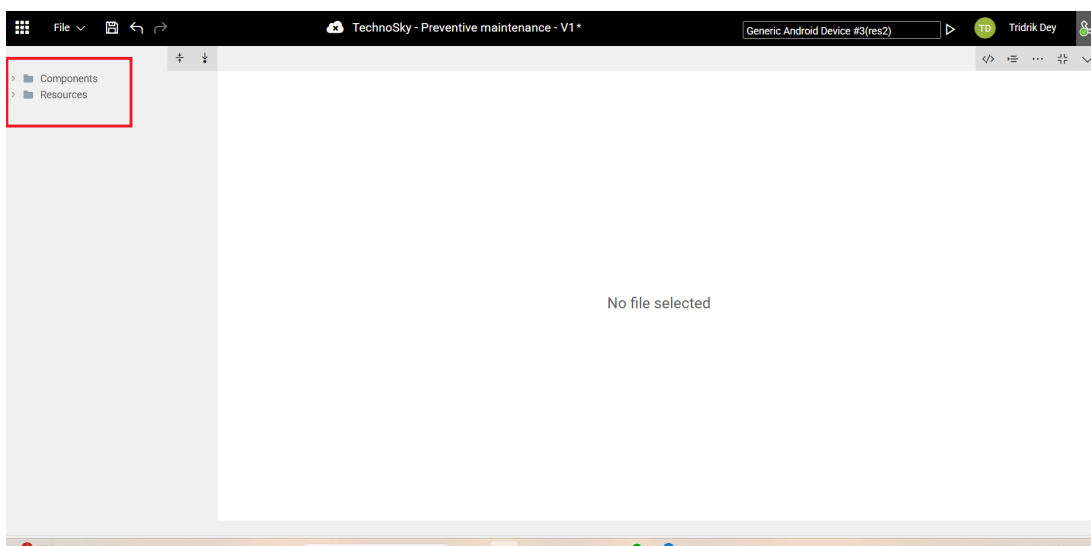
1. L'utente seleziona scansione o elenco
2. Il sistema recupera l'ordine
3. Inizializza tutti i parametri dell'ordine
4. Tenta l'estrazione dinamica del flusso di lavoro tramite l'ID del piano di lavoro
5. Continua con o senza modello inserito

PASSAGGI PER EFFETTUARE TRADUZIONI

Guarda lo screenshot. Se clicchi sull'icona in rosso o sul CODE, si apre l'editor di codice avanzato.

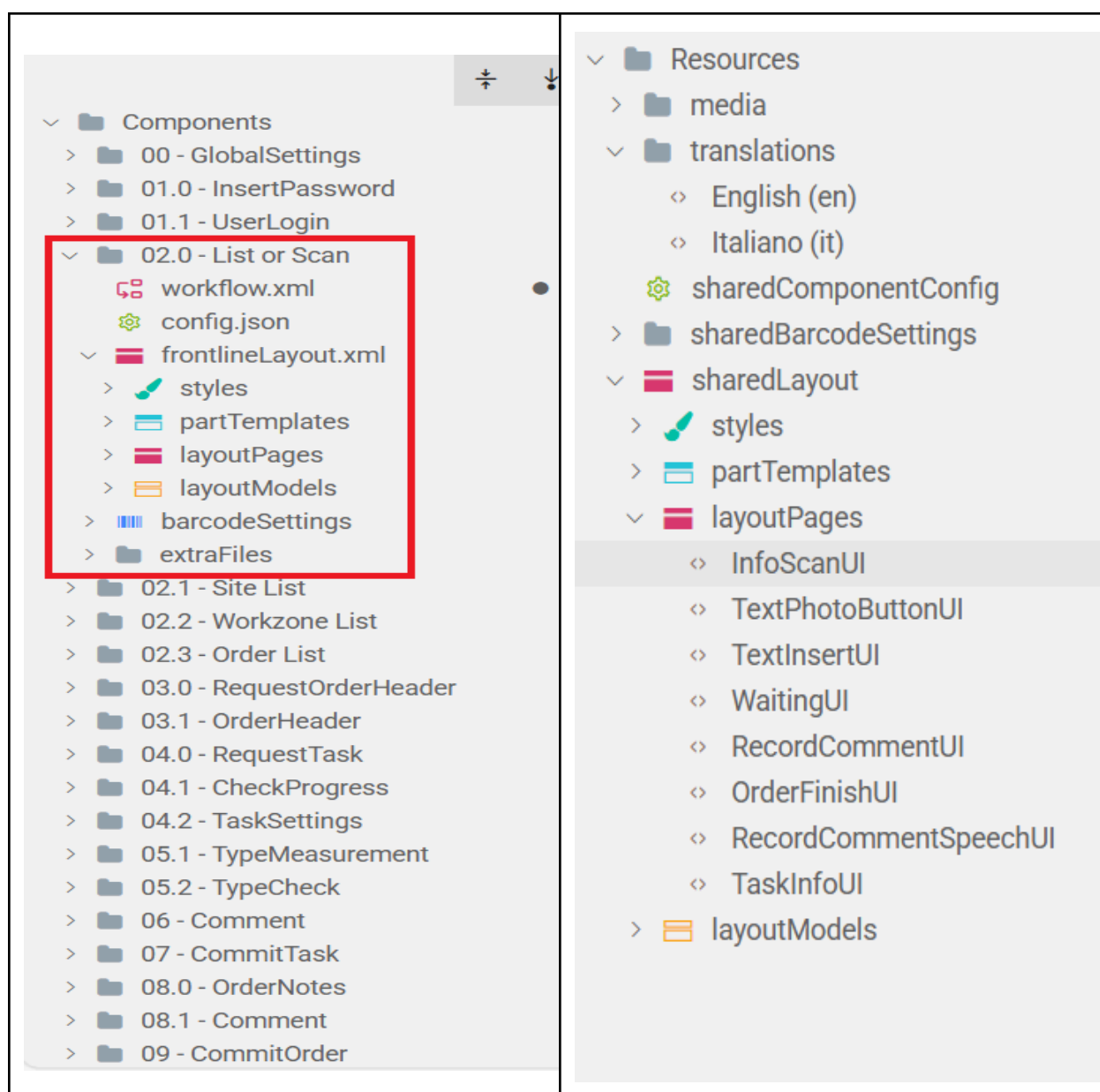


Code Editor: Nell'editor del codice ci sono due cartelle: Components e Resources.



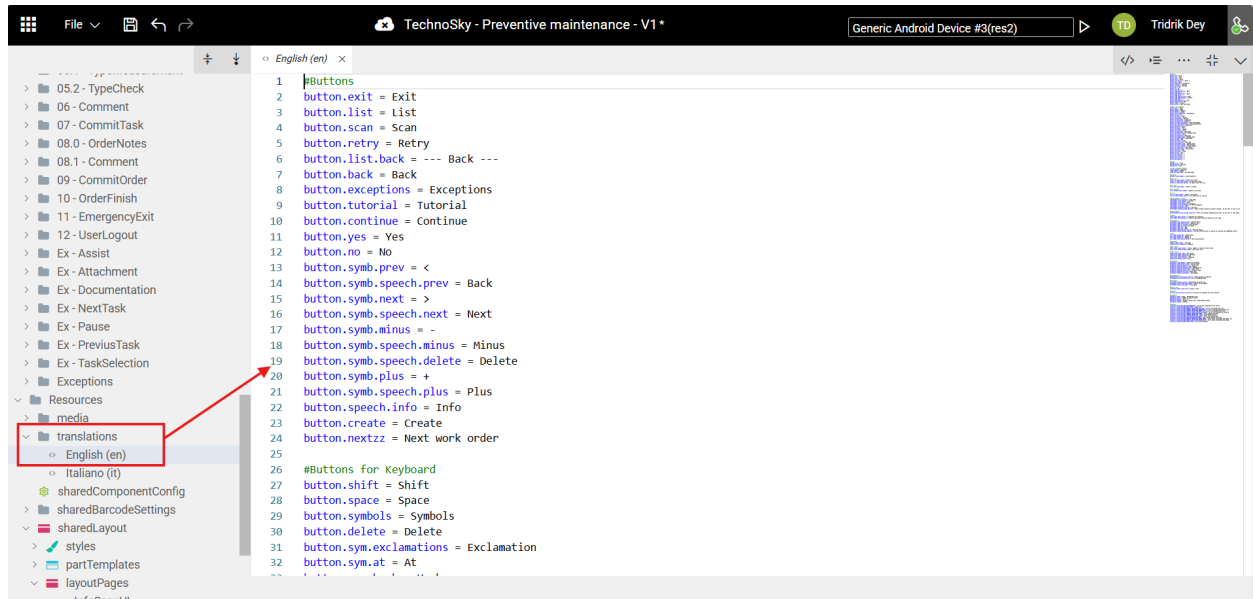
La cartella dei componenti contiene ciascun componente presente nell'architettura di rete, collegato da frecce. Ogni cartella dei componenti avrà il proprio flusso di lavoro, file di configurazione e layout.

La cartella Risorse conterrà elementi comuni a tutti i componenti. Include Media, Traduzioni, Config condivisa e Layout condiviso.

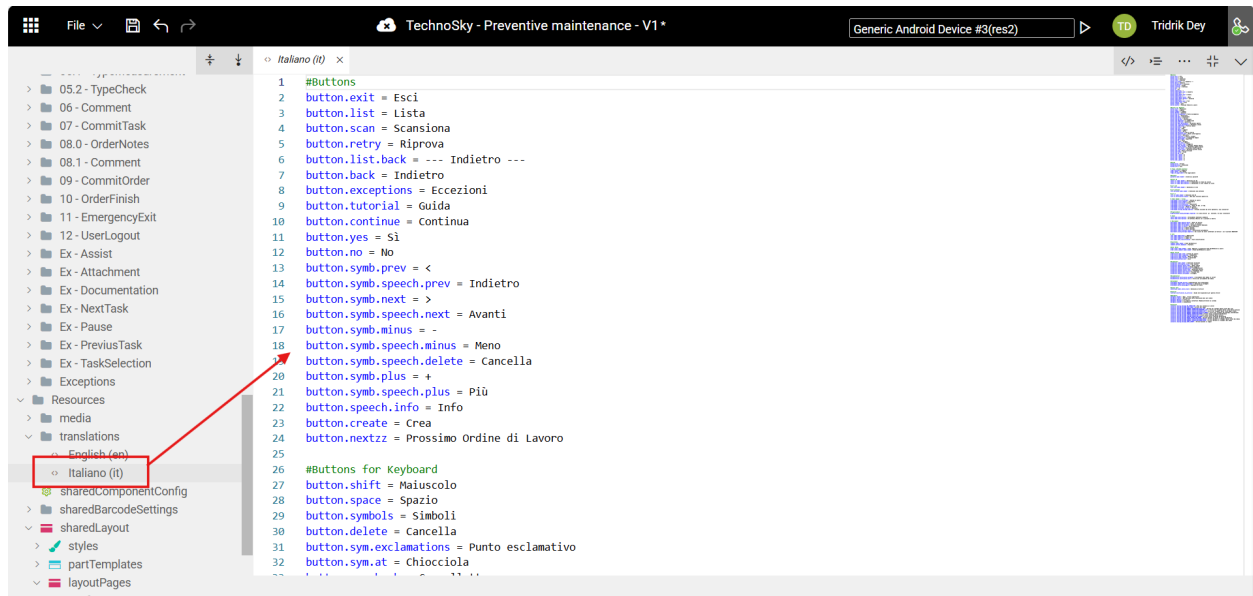


TRADUZIONI

Inglese:



Italiano:



All'interno di ogni workflow possono esserci casi in cui il testo dell'interfaccia utente (UI) non è scritto in modo hardcoded nel layout o nella logica applicativa.

In questi casi vengono utilizzate **chiavi di traduzione** per garantire il supporto multilingua.

Ad esempio:

```
<ui_update id="show_buttons">
  <widget_params>
    <ui_element name="1BTN">
      <param name="visibility">Visible</param>
    </ui_element>
    <ui_element name="1BTN_TEXT">
      <param name="content">${button.back}</param>
    </ui_element>
    <ui_element name="2BTN">
      <param name="visibility">Visible</param>
    </ui_element>
    <ui_element name="2BTN_TEXT">
      <param name="content">${button.continue}</param>
    </ui_element>
  </widget_params>
</ui_update>
```

Nel file di traduzione presente nella cartella resources, la chiave viene definita per ciascuna lingua:

Italiani:	Inglese:
<pre>#Buttons button.exit = Esci button.list = Lista button.scan = Scansione button.retry = Riprova button.list.back = --- Indietro --- button.back = Indietro button.exceptions = Eccezioni button.tutorial = Guida button.continue = Continua button.yes = Sì button.no = No button.symb.prev = < button.symb.speech.prev = Indietro button.symb.next = > button.symb.speech.next = Avanti button.symb.minus = -</pre>	<pre>#Buttons button.exit = Exit button.list = List button.scan = Scan button.retry = Retry button.list.back = --- Back --- button.back = Back button.exceptions = Exceptions button.tutorial = Tutorial button.continue = Continue button.yes = Yes button.no = No button.symb.prev = < button.symb.speech.prev = Back button.symb.next = > button.symb.speech.next = Next button.symb.minus = -</pre>

Per modificare il testo visualizzato nella UI, è necessario aggiornare la relativa chiave nel file di traduzione appropriato all'interno della cartella resources.

Il sistema visualizza dinamicamente il testo in base alla lingua selezionata. Pertanto:

- Se è impostata la lingua inglese → verrà mostrato "Continue"
- Se è impostata la lingua italiana → verrà mostrato "Continua"

È importante assicurarsi che tutti i testi dell'interfaccia destinati al supporto multilingua siano gestiti tramite chiavi di traduzione e non inseriti direttamente in modo hardcoded.